

1. SELECT Y ORDEN LOGICO

```
SELECT [DISTINCT] col1, col2 AS alias
FROM tabla t
WHERE condicion_filas
GROUP BY columnas
HAVING condicion_grupos
ORDER BY col1 DESC
FETCH FIRST 10 ROWS ONLY;
```

| Parte    | Uso                           |
|----------|-------------------------------|
| WHERE    | Filtra filas antes de agrupar |
| GROUP BY | Agrupar filas                 |
| HAVING   | Filtra despues de agrupar     |
| ORDER BY | Orden final                   |

2. FILTROS, OPERADORES, LIKE

= <> > < >= <= AND OR NOT

```
WHERE campo_num BETWEEN 100 AND 500
WHERE estado IN ('A','B','C')
WHERE texto LIKE 'AB%' -- empieza
WHERE texto LIKE '%Z' -- termina
WHERE campo IS NULL
WHERE campo IS NOT NULL
```

LIKE: % varios caracteres, ☐ un caracter. Textos entre comillas simples.

3. FUNCIONES FRECUENTES

| Tipo       | Funciones                                          |
|------------|----------------------------------------------------|
| Texto      | UPPER, LOWER, INITCAP, LENGTH, SUBSTR, INSTR, TRIM |
| Numero     | ROUND, TRUNC, MOD, ABS, CEIL, FLOOR                |
| Fechas     | SYSDATE, ADD_MONTHS, MONTHS_BETWEEN, EXTRACT       |
| Nulos      | NVL, NVL2, COALESCE, NULLIF                        |
| Conversion | TO_CHAR, TO_DATE, TO_NUMBER                        |

```
SELECT UPPER(texto), LENGTH(texto)
FROM tabla;

WHERE fecha >= DATE '2026-01-01'
WHERE fecha = TO_DATE('31/12/2026', 'DD/MM/YYYY')
```

4. AGREGACION

| Funcion    | Devuelve        |
|------------|-----------------|
| COUNT(*)   | N. filas        |
| COUNT(col) | No cuenta NULL  |
| SUM / AVG  | Suma / media    |
| MIN / MAX  | Minimo / maximo |

```
SELECT categoria, ROUND(AVG(importe),2)
media
FROM tabla
GROUP BY categoria
HAVING AVG(importe) > 1000;
```

Regla: toda columna normal del SELECT debe estar en GROUP BY si hay funciones de grupo.

5. JOINS

| JOIN  | Idea                     |
|-------|--------------------------|
| INNER | Solo filas con pareja    |
| LEFT  | Conserva tabla izquierda |
| RIGHT | Conserva tabla derecha   |
| FULL  | Conserva ambos lados     |
| SELF  | Tabla consigo misma      |

```
SELECT a.nombre, b.fecha
FROM tabla_a a
JOIN tabla_b b ON b.id_a = a.id_a;

SELECT x.nombre, y.nombre padre
FROM tabla_x x
LEFT JOIN tabla_x y ON x.id_padre=y.id_x;
```

NATURAL JOIN: une por columnas con mismo nombre.  
Usarlo solo si estas seguro.

6. SUBCONSULTAS

```
-- Subconsulta que devuelve un valor
SELECT nombre
FROM tabla
WHERE importe = (SELECT MAX(importe) FROM
tabla);

-- Lista de valores
WHERE id IN (SELECT id FROM otra_tabla);

-- Correlada / EXISTS
WHERE EXISTS (
  SELECT 1 FROM tabla_b b
  WHERE b.id_a = a.id_a
);
```

| Operador  | Cuándo                       |
|-----------|------------------------------|
| =, <, >   | Subconsulta de 1 valor       |
| IN        | Lista de valores             |
| EXISTS    | Si basta con que exista fila |
| ANY / ALL | Comparar con varios valores  |

7. CASE Y ALIAS

```
SELECT nombre,
CASE
  WHEN importe >= 1000 THEN 'ALTO'
  WHEN importe >= 500 THEN 'MEDIO'
  ELSE 'BAJO'
END AS tramo
FROM tabla;
```

8. DML Y TRANSACCIONES

```
INSERT INTO tabla (id, texto, importe)
VALUES (1, 'ABC', 100);

UPDATE tabla
SET importe = importe * 1.05
WHERE categoria = 'X';

DELETE FROM tabla
WHERE estado = 'ANULADO';

SAVEPOINT p1; COMMIT; ROLLBACK;
```

Cuidado: UPDATE/DELETE sin WHERE afectan a toda la tabla.

9. DDL Y RESTRICCIONES

```
CREATE TABLE tabla_a (
  id_a NUMBER(4) PRIMARY KEY,
  texto VARCHAR2(40) NOT NULL,
  importe NUMBER(8,2),
  id_b NUMBER(4),
  CONSTRAINT tabla_a_fk
  FOREIGN KEY (id_b) REFERENCES
  tabla_b(id_b),
  CONSTRAINT tabla_a_ck
  CHECK (importe >= 0),
  CONSTRAINT tabla_a_uk
  UNIQUE (texto)
);

ALTER TABLE tabla_a ADD campo VARCHAR2(20);
ALTER TABLE tabla_a DROP COLUMN campo;
DROP TABLE tabla_a CASCADE CONSTRAINTS;
```

| Tipo Oracle | Uso            |
|-------------|----------------|
| NUMBER(p,s) | Numeros        |
| VARCHAR2(n) | Texto variable |
| CHAR(n)     | Texto fijo     |
| DATE        | Fecha y hora   |

10. MODELO RELACIONAL DESDE E/R

| E/R                   | Paso a tablas                    |
|-----------------------|----------------------------------|
| Entidad fuerte        | Tabla con su PK                  |
| 1:N                   | FK en el lado N                  |
| N:M                   | Tabla intermedia con 2 FK        |
| Atributo multivaluado | Tabla aparte: PK entidad + valor |
| Recursiva 1:N         | FK a la propia tabla             |
| Debil                 | PK propia + PK de entidad fuerte |
| Especializacion       | Supertipo + subtipos (PK=FK)     |

FK opcional: admite NULL si la participacion no es obligatoria.

11. NORMALIZACION

| FN  | Idea clave                                    |
|-----|-----------------------------------------------|
| 1FN | Atributos atomicos; sin listas en una celda   |
| 2FN | Sin dependencias parciales de clave compuesta |
| 3FN | Sin dependencias transitivas entre no claves  |

```
-- Multivaluado generico
ENTIDAD(id, nombre, telefono1, telefono2)
-- Mejor:
ENTIDAD(id, nombre)
ENTIDAD_TELEFONO(id, telefono)
```

12. CHECKLIST SQL

- ¿El SELECT tiene FROM?
- ¿Los textos van con comillas simples?
- ¿Las fechas usan DATE o TO\_DATE?
- ¿Las columnas agregadas respetan GROUP BY?
- ¿La subconsulta devuelve 1 valor si usas =?
- ¿JOIN con condicion ON correcta?
- ¿UPDATE/DELETE tienen WHERE?
- ¿PK subrayada y FK identificada en modelo relacional?

13. BLOQUE ANONIMO

```
SET SERVEROUTPUT ON;
DECLARE
  v_total NUMBER := 0;
  v_texto VARCHAR2(40);
BEGIN
  SELECT COUNT(*) INTO v_total
  FROM tabla;
  DBMS_OUTPUT.PUT_LINE('Total: ' || v_total);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE(SQLERRM);
END
```

| Zona      | Qué va                     |
|-----------|----------------------------|
| DECLARE   | Variables, cursores, tipos |
| BEGIN     | Instrucciones              |
| EXCEPTION | Tratamiento de errores     |
| END; /    | Fin y ejecucion            |

14. VARIABLES, TIPOS Y SALIDA

```
v_num NUMBER(8,2) := 0;
v_txt VARCHAR2(50);
v_fecha DATE := SYSDATE;
v_col tabla.campo%TYPE;
r_fila tabla%ROWTYPE;
```

```
DBMS_OUTPUT.PUT_LINE(v_txt || ' - ' || v_num);
```

| Recurso  | Uso                                         |
|----------|---------------------------------------------|
| %TYPE    | Mismo tipo que una columna                  |
| %ROWTYPE | Registro con columnas de una tabla o cursor |
|          | Concatenar texto                            |
| :=       | Asignar valor PL/SQL                        |

15. SELECT INTO Y ERRORES

```
DECLARE
  v_importe tabla.importe%TYPE;
BEGIN
  SELECT importe INTO v_importe
  FROM tabla
  WHERE id = 1;
EXCEPTION
  WHEN NO_DATA_FOUND THEN ...
  WHEN TOO_MANY_ROWS THEN ...
END
```

SELECT INTO debe devolver exactamente una fila. Para varias filas, usa cursor.

16. IF, CASE, BUCLES

```
IF v_importe >= 1000 THEN
  ...
ELSIF v_importe >= 500 THEN
  ...
ELSE
  ...
END IF;

CASE v_estado
  WHEN 'A' THEN ...
  ELSE ...
END CASE;

FOR i IN 1..10 LOOP ... END LOOP;
WHILE condicion LOOP ... END LOOP;
```

17. CURSOR FOR SIMPLE

```
DECLARE
  CURSOR c_datos IS
    SELECT id, texto, fecha
    FROM tabla
    WHERE condicion
    ORDER BY fecha;
BEGIN
  FOR r IN c_datos LOOP
    DBMS_OUTPUT.PUT_LINE(r.id || ' * ' || r.texto);
  END LOOP;
END;
```

El FOR abre, lee y cierra automaticamente. Suele ser la forma mas limpia.

18. CURSOR MANUAL

```
DECLARE
  CURSOR c IS SELECT id, importe FROM tabla;
  r c%ROWTYPE;
BEGIN
  OPEN c;
  FETCH c INTO r;
  WHILE c%FOUND LOOP
    DBMS_OUTPUT.PUT_LINE(r.id);
    FETCH c INTO r;
  END LOOP;
  CLOSE c;
END;
```

| Atributo  | Significa                  |
|-----------|----------------------------|
| %FOUND    | Ultimo FETCH encontro fila |
| %NOTFOUND | No encontro fila           |
| %ROWCOUNT | Filas leidas               |
| %ISOPEN   | Cursor abierto             |

19. CURSOR FOR UPDATE

```
DECLARE
  CURSOR c IS
    SELECT importe
    FROM tabla
    FOR UPDATE;
  v_cambio NUMBER;
BEGIN
  FOR r IN c LOOP
    IF r.importe >= 1000 THEN
      v_cambio := r.importe * 0.10;
    ELSE
      v_cambio := 0;
    END IF;

    UPDATE tabla
    SET importe = importe - v_cambio
    WHERE CURRENT OF c;
  END LOOP;
END;
```

20. PROCEDIMIENTOS

```
CREATE OR REPLACE PROCEDURE p_nombre (
  p_id IN NUMBER,
  p_texto IN VARCHAR2,
  p_total OUT NUMBER
) AS
BEGIN
  UPDATE tabla
  SET texto = p_texto
  WHERE id = p_id;

  SELECT COUNT(*) INTO p_total FROM tabla;
END;
```

| Parametro | Uso                       |
|-----------|---------------------------|
| IN        | Entra al procedimiento    |
| OUT       | Sale como resultado       |
| IN OUT    | Entra y puede modificarse |

21. FUNCIONES

```
CREATE OR REPLACE FUNCTION f_total
RETURN NUMBER IS
  v_total NUMBER;
BEGIN
  SELECT SUM(importe) INTO v_total
  FROM tabla;
  RETURN NVL(v_total,0);
END;
```

La funcion siempre debe devolver un valor con RETURN.

22. TRIGGERS

```
CREATE OR REPLACE TRIGGER t_nombre
BEFORE INSERT OR UPDATE OF campo ON tabla
FOR EACH ROW
BEGIN
  IF :NEW.campo < 0 THEN
    RAISE_APPLICATION_ERROR(-20001,
      'Valor no permitido');
  END IF;

  :NEW.texto := UPPER(TRIM(:NEW.texto));
END;
```

| Elemento     | Uso                |
|--------------|--------------------|
| :OLD         | Valor anterior     |
| :NEW         | Valor nuevo        |
| BEFORE       | Antes del cambio   |
| AFTER        | Despues del cambio |
| FOR EACH ROW | Trigger por fila   |

23. ERRORES Y RECORDATORIOS

| Error            | Idea                           |
|------------------|--------------------------------|
| NO_DATA_FOUND    | SELECT INTO sin filas          |
| TOO_MANY_ROWS    | SELECT INTO devuelve varias    |
| DUP_VAL_ON_INDEX | PK/UNIQUE repetida             |
| VALUE_ERROR      | Conversion o tamano incorrecto |
| OTHERS           | Cualquier otro error           |

- Termina sentencias con `;`.
- Termina bloques con `END;` y ejecuta con `/`.
- En PL/SQL se asigna con `:=`, en SQL se compara con `=`.
- En trigger de fila usa `:NEW` y `:OLD`.
- Si recorres muchas filas, usa cursor o cursor FOR.
- Cuida los nombres: columnas, alias, parametros y variables.

24. PLANTILLAS RAPIDAS

```
-- Mostrar filas:
FOR r IN (SELECT ... FROM ...) LOOP
  DBMS_OUTPUT.PUT_LINE(r.campo);
END LOOP;

-- Validar:
IF condicion no valida THEN
  RAISE_APPLICATION_ERROR(-20002, 'Mensaje');
END IF;
```